

Veridise
Auditing Report

Hardening Blockchain Security with Formal Methods

FOR



Falcon Smart Account for Soroban



Veridise Inc.
August 12, 2026

► **Prepared For:**

Soundness Labs

<https://www.soundness.xyz>

► **Prepared By:**

Alp Bassa

Alberto Gonzalez

Daniel Dominguez

► **Contact Us:**

contact@veridise.com

► **Version History:**

August 24, 2026 V1

September 10, 2026 V2

Contents

Contents	iii
1 Executive Summary	1
2 Project Dashboard	4
3 Security Assessment Goals and Scope	5
3.1 Security Assessment Goals	5
3.2 Security Assessment Methodology & Scope	5
3.3 Classification of Vulnerabilities	6
4 Trust Model	7
4.1 Operational Assumptions	7
4.2 Privileged Roles	7
5 Vulnerability Report	8
5.1 Detailed Description of Issues	9
5.1.1 V-FSA-VUL-001: Non-standard 0x29 detached Falcon signatures are accepted	9
5.1.2 V-FSA-VUL-002: Unsafe key rotation may permanently lock smart accounts	11
5.1.3 V-FSA-VUL-003: Non-standard Falcon input limits may impair interoperability	12
5.1.4 V-FSA-VUL-004: Undocumented NTT primitives representation invariants may cause incorrect computations	14
5.1.5 V-FSA-VUL-005: Maintainability and Documentation Findings	16
6 Fuzz Testing	18
6.1 Methodology	18
6.2 Properties Fuzzed	18

About Veridise

Veridise is an independent security firm founded by academics and security researchers with deep expertise in formal methods, programming languages, and applied cryptography. The firm focuses on high-assurance security for blockchain and cryptographic systems, combining rigorous theory with practical adversarial analysis.

Veridise provides full-stack blockchain security services spanning smart contracts, zero-knowledge circuits and proving systems, consensus and execution clients, cryptographic libraries, and supporting infrastructure. The team routinely analyzes systems at the boundary between protocol design, implementation, and cryptography, where failures are both subtle and high-impact. To date, Veridise has conducted hundreds of security assessments for protocols securing billions of dollars in TVL.

Dynamic Security Veridise treats security as an ongoing engineering discipline rather than a one-time validation exercise. Effective security requires iterative review, continuous feedback, and close collaboration between auditors and system designers. Throughout the engagement, Veridise analysts communicated regularly with the Soundness Labs team using [AuditHub](#), enabling rapid clarification of design intent, prompt discussion of findings, and efficient validation of fixes.

In parallel with manual review, Veridise maintains active research programs focused on automated and formal analysis of blockchain systems. These efforts have produced tools such as *Vanguard* for static analysis and *Picus* for formal verification, which are available through [AuditHub](#). AuditHub integrates these techniques directly into CI/CD workflows, supporting continuous vulnerability detection and regression analysis beyond the scope of a single audit.

Additional details and case studies are available at <https://audithub.dev/case-studies/>.

Veridise Reports Veridise reports are published in the public audit archive linked below. Only reports obtained from this archive or confirmed directly by a representative of [Veridise](#) should be considered official Veridise reports.

<https://veridise.com/audits-archive/>

Executive Summary

From Aug. 12, 2026 to Aug. 17, 2026, Soundness Labs engaged Veridise to conduct a security assessment of their Falcon Smart Account for Soroban*. The security assessment covered the implementation of a Stellar contract account† that authenticates authorization contexts using Falcon-512, a post-quantum signature scheme.

Veridise conducted the assessment over 8 person-days, with 2 security analysts reviewing the project over 4 days on commit 79d8c99. The review strategy involved thorough manual code review by Veridise security analysts and differential fuzzing against other Falcon-512 implementations by a dedicated fuzzing engineer.

Project Summary. The project is organized into three crates:

- ▶ `falcon-512-core`. A `no_std`, host-independent verifier that implements the Falcon-512 verification algorithm including the implementation of the ring arithmetic primitives such as field operations over $q = 12289$, Montgomery multiplication, and efficient polynomial evaluation and multiplication using the number-theoretic transform (NTT).
- ▶ `soroban-falcon-smart-account`. A Soroban contract account which stores a Falcon-512 public key, supports key rotation, and, in `__check_auth`, verifies a Falcon signature over a domain-separated authorization payload using the `falcon-512-core` crate.
- ▶ `soroban-falcon-verifier`. A wrapper around `falcon-512-core`. It exposes a public `verify(pk, msg, sig)` entry point and contains no additional cryptographic logic.

Code Assessment. The Falcon Smart Account for Soroban developers provided the source code of the Falcon Smart Account for Soroban contracts for the code review. The Soroban contract wrappers appear to be original. The Falcon-512 verification core is a Rust implementation of the NIST Round-3 verifier that uses PQClean `falcon-512/clean`‡ as a reference for the ring arithmetic (field operations, Montgomery multiplication, and the NTT).

To facilitate the Veridise security analysts' understanding of the code, the Falcon Smart Account for Soroban developers shared README files, rustdoc, and an audit pack covering the threat model, verification design, and KAT vectors.

The source code contained a test suite, which the Veridise security analysts noted covered unit and integration tests of the verification pipeline, including all 100 official NIST Round-3 Falcon-512 KAT vectors.

Summary of Issues Detected. The security assessment uncovered 5 issues, none of which the Veridise analysts assessed as high or critical severity. Of these, they identified 1 medium-severity issue. Specifically, [V-FSA-VUL-001](#) shows that the verifier conflates distinct Falcon signature formats by accepting the NIST signed-message header `0x29` as a detached signature header alongside the standard `0x39`. This creates a non-standard, malleable encoding and contradicts the documented claim that Falcon signers disagree on the meaning of these headers.

* <https://github.com/SoundnessLabs/stellar-pq/>

† <https://developers.stellar.org/docs/build/guides/contract-accounts>

‡ <https://github.com/PQClean/PQClean>

In addition 2 low-severity issues were identified. Issue [V-FSA-VUL-002](#) shows that insufficiently validated, single-step key rotation can irreversibly install a malformed or unintended public key, permanently rendering the wallet unusable. Issue [V-FSA-VUL-003](#) reveals that the verifier's message, signature-length, and coefficient limits diverge from standard Falcon-512 encodings, rejecting valid inputs and creating interoperability and availability risks. In addition, 2 warnings were found.

Recommendations. After conducting the assessment of the protocol, the security analysts had a few suggestions to improve the Falcon Smart Account for Soroban codebase.

Extend Inline Documentation for Arithmetic Assumptions. The Veridise analysts recommend extending the inline documentation for the Falcon arithmetic code, especially `ntt.rs`. The NTT and field helper functions should document their mathematical purpose, expected input range, expected representation, and output guarantees, including whether values are canonical modulo Q , in coefficient form, in evaluation form, or in Montgomery form. Constants that differ from common Falcon or implementation-level conventions should also include a short explanation of their role and justification. This would make the code easier to maintain and reduce the chance of future changes violating representation or range assumptions.

Encode Field and Polynomial Representations in the Type System. The NTT primitives in `ntt.rs` currently represent field elements as `u32` and polynomials as `u16` arrays, regardless of whether the polynomial is in coefficient form, NTT/evaluation form, natural or Montgomery encoded. As a result, the compiler cannot check important assumptions such as whether a field element is reduced modulo Q whether a field operation returns a reduced value, or whether a function is receiving a polynomial in the representation it expects. This makes mistakes such as passing a coefficient-form polynomial to a function that expects NTT form, multiplying values before converting to Montgomery form, applying the Montgomery conversion twice, or forgetting a required conversion possible during future refactors. Introduce custom types that encode these assumptions directly: a field-element type whose private value is always canonical modulo Q , with arithmetic operations that also return canonical values, and separate polynomial types for the relevant representations used by the NTT code. This lets each function express its expected input and output representations in its signature. The benefit is that Rust's type checker can catch representation mismatches at compile time, making missing, duplicated, or incorrectly ordered conversions much harder to introduce.

Extend Arithmetic and Parser Test Coverage. The Veridise analysts recommend extending the test suite for the Falcon arithmetic code and for the public-key and signature decoding paths. The field operations in `ntt.rs` should be checked exhaustively against a simple independent modular-arithmetic oracle. Since $Q = 12289$ is small, addition, subtraction, halving, Montgomery multiplication, and representation conversions can be tested over the full relevant input range rather than only through sampled cases. Polynomial tests should also cover forward/inverse NTT round trips, comparison against an independent schoolbook negacyclic multiplication implementation, and checks that coefficients remain canonical after each transformation. The Montgomery constants and twiddle tables should be tested by regenerating them from their documented mathematical definitions and comparing every entry, which would catch accidental table corruption. Finally, parser tests should cover malformed encodings such as invalid public-key coefficients, negative zero, magnitudes at the 2047/2048 boundary, truncation at different bit positions, nonzero unused bits, malformed padding, and alternate headers. Together, these tests would make arithmetic, representation-conversion, and decoding regressions much easier to detect during future refactors.

Disclaimer. We hope that this report is informative but provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the system is secure in all dimensions. In no event shall Veridise or any of its employees be liable for any claim, damages or other liability, whether in an action of contract, tort or otherwise, arising from, out of or in connection with the results reported here.

Table 2.1: Application Summary.

Name	Version	Type	Platform
Falcon Soroban Smart Account	79d8c99	Rust	Stellar

Table 2.2: Engagement Summary.

Dates	Method	Consultants Engaged	Level of Effort
Aug. 12–Aug. 17, 2026	Manual & Tools	2	8 person-days

Table 2.3: Vulnerability Summary.

Name	Number	Acknowledged	Fixed
Critical-Severity Issues	0	0	0
High-Severity Issues	0	0	0
Medium-Severity Issues	1	1	1
Low-Severity Issues	2	2	2
Warning-Severity Issues	2	2	2
Informational-Severity Issues	0	0	0
TOTAL	5	5	5

Table 2.4: Category Breakdown.

Name	Number
Data Validation	2
Maintainability	2
Access Control	1



3.1 Security Assessment Goals

The engagement was scoped to provide a security assessment of Falcon Smart Account for Soroban's source code. During the assessment, the security analysts aimed to answer questions such as:

- ▶ Is CustomAccountInterface implemented correctly? In particular, does `__check_auth` accept authorization if and only if the supplied Falcon-512 signature verifies under the stored public key over the host-provided `signature_payload`?
- ▶ Can privileged operations, such as key rotation, be performed without a valid Falcon-512 authorization from the account?
- ▶ Can signatures produced for a different context be replayed against the smart account?
- ▶ Does the verifier accept malleable encodings of a valid signature, and if so, can that be used to replay a previous authorization?
- ▶ Are public-key and signature encodings parsed strictly and canonically? Are supported signature formats and length limits correct?
- ▶ Is hash-to-point correct, unbiased, and bound to the right transcript?
- ▶ Is the finite-field representation and arithmetic in $\mathbb{Z}_{12289}$ implemented correctly and safely across all representations? Are field elements always represented canonically in $[0, q - 1]$, with $q = 12289$?
- ▶ Are polynomial representations, operations, and NTT implemented correctly?
- ▶ Is the transition to and from Montgomery representation and Montgomery reduction implemented correctly?
- ▶ Is transformation between signed and unsigned representation, norm computation, and the corresponding overflow handling sound?

3.2 Security Assessment Methodology & Scope

Security Assessment Methodology. To address the questions above, the security assessment involved a combination of human experts and automated program analysis & testing tools. In particular, the security assessment was conducted with the aid of the following techniques:

- ▶ *Manual review.* The Veridise analysts performed a line-by-line review of the in-scope code and supporting documentation to evaluate design assumptions and identify logic flaws.
- ▶ *Unit Testing.* The Veridise analysts exhaustively checked the verifier's field arithmetic such as the Montgomery multiplication, addition, subtraction, and halving operations against a naive specification over $\mathbb{Z}_q$, covering every canonical input in $[0, q - 1]$ and, for binary operations, every pair in $\mathbb{Z}_q^2$. They also compared the verifier's NTT-based multiplication in $\mathbb{Z}_q[X]/(X^{512} + 1)$ against an independent implementation based on the arkworks library.

- *Fuzzing/Property-based Testing.* A dedicated fuzzing engineer used AFL++ to fuzz the Falcon-512 verifier against two properties: that verification does not panic on correctly sized inputs, and that it agrees with the audited `falcon-rs` reference on whether a given signature is valid. The differential check was run both on arbitrary correctly sized inputs and on valid signatures generated by the reference implementation. Further details about the fuzzing methodology can be found in [Chapter 6](#).

Scope. The scope of this security assessment is limited to the following crates under contracts/: `falcon-512-core`, `soroban-falcon-smart-account` and `soroban-falcon-verifier`.

Methodology. Veridise security analysts reviewed the developer-provided audit pack, including its threat model and supporting security materials, inspected the provided tests, and read the project documentation. They also consulted Falcon-512 reference implementations and the corresponding specification when evaluating the verifier.

3.3 Classification of Vulnerabilities

When Veridise security analysts discover a possible security vulnerability, they must estimate its severity by weighing its potential impact against the likelihood that a problem will arise.

The severity of a vulnerability is evaluated according to the [Table 3.1](#).

Table 3.1: Severity Breakdown.

	Somewhat Bad	Bad	Very Bad	Protocol Breaking
Not Likely	Info	Warning	Low	Medium
Likely	Warning	Low	Medium	High
Very Likely	Low	Medium	High	Critical

The likelihood of a vulnerability is evaluated according to the [Table 3.2](#).

Table 3.2: Likelihood Breakdown

Not Likely	A small set of users must make a specific mistake
Likely	Requires a complex series of steps by almost any user(s) - OR - Requires a small set of users to perform an action
Very Likely	Can be easily performed by almost anyone

The impact of a vulnerability is evaluated according to the [Table 3.3](#):

Table 3.3: Impact Breakdown

Somewhat Bad	Inconveniences a small number of users and can be fixed by the user
Bad	Affects a large number of people and can be fixed by the user - OR - Affects a very small number of people and requires aid to fix
Very Bad	Affects a large number of people and requires aid to fix - OR - Disrupts the intended behavior of the protocol for a small group of users through no fault of their own
Protocol Breaking	Disrupts the intended behavior of the protocol for a large group of users through no fault of their own



4.1 Operational Assumptions

Falcon Smart Account for Soroban has no ongoing protocol-operator actions other than the Falcon public-key rotation. Analysts assumed that `rotate_key` is invoked conscientiously: the signer supplies a new Falcon-512 public key that they control. A failure in that process, for example, rotating to a key the user does not control is treated as operational failures rather than a protocol bug.

4.2 Privileged Roles

Falcon Smart Account for Soroban is a single-signer smart account. There are no administrators, pausers, upgraders, or other operator keys. The only privileged entity is the holder of the Falcon-512 private key corresponding to the public key stored in the contract.

In particular, the holder of the Falcon-512 private key can authorize any invocation as the smart account, including transfers of account assets and `rotate_key`. The contract account `__check_auth` verifies only that the supplied Falcon signature is valid over the domain-separated host payload. It does not inspect `_auth_contexts` and does not otherwise restrict which calls may be authorized. A valid signature therefore grants the signer unbounded authority to authorize arbitrary contract calls on behalf of the contract account.

Operational Recommendations. Compromise of the Falcon signer is equivalent to full control of the account. Users should:

- ▶ Never store the Falcon seed in plaintext, on a regularly used phone, laptop, or device, or in a custom key-management solution that has not been independently reviewed.
- ▶ Prefer a dedicated credential store (hardware token, secure enclave, or equivalent) for the seed.
- ▶ Treat `rotate_key` as a sensitive operation: confirm the new public key before rotation, and rotate immediately after any suspected compromise.
- ▶ Securely store physical, non-digital backups of the seed.
- ▶ Monitor for unexpected invocations authorized by the account.

Full validation of operational security practices is beyond the scope of this review.

5

Vulnerability Report

This section presents the vulnerabilities found during the security assessment. For each issue found, the type of the issue, its severity, location in the code base, and its current status (i.e., acknowledged, fixed, etc.) is specified. Table 5.1 summarizes the issues discovered:

Table 5.1: Summary of Discovered Vulnerabilities.

ID	Description	Severity	Status
V-FSA-VUL-001	Non-standard 0x29 detached Falcon . . .	Medium	Fixed
V-FSA-VUL-002	Unsafe key rotation may permanently lock . . .	Low	Fixed
V-FSA-VUL-003	Non-standard Falcon input limits may . . .	Low	Fixed
V-FSA-VUL-004	Undocumented NTT primitives . . .	Warning	Fixed
V-FSA-VUL-005	Maintainability and Documentation Findings	Warning	Fixed

5.1 Detailed Description of Issues

5.1.1 V-FSA-VUL-001: Non-standard 0x29 detached Falcon signatures are accepted

Severity	Medium	Commit	79d8c99
Type	Data Validation	Status	Fixed
Location(s)	contracts/falcon-512-core/src/verify.rs:101-120		
Confirmed Fix At	stellar-pq/pull/2		

Description The verifier accepts both 0x29 and 0x39 as headers for the same detached signature layout:

```
1 header || nonce[40] || compressed(s2)
```

After checking that the high nibble is either 0x2X or 0x3X, the verifier always extracts bytes 1-40 as the nonce and invokes the compressed polynomial decoder. The selected header family therefore has no effect on framing or decoding.

This conflates two distinct formats defined by Falcon:

- ▶ 0x39 identifies a Falcon-512 compressed signature containing the nonce. Both natural-length and 666-byte padded signatures use this header.
- ▶ 0x59 identifies the alternate fixed-width/constant-time encoding and requires a different decoder.
- ▶ 0x29 is used only for the nonce-less signature tail inside the NIST `crypto_sign` signed-message envelope:

Consequently, the local representation `0x29 || nonce || compressed(s2)` is a non-standard hybrid format.

The comments claiming that signers disagree about the meanings of these nibbles are inaccurate. The Falcon specification, reference C implementation, PQCclean, and the Python signer agree on the relevant conventions:

- ▶ Detached compressed and padded Falcon-512 signatures use 0x39.
- ▶ The CT/fixed-width representation uses 0x59.
- ▶ 0x29 belongs to the nonce-less tail of the NIST signed-message envelope.

The KAT tests currently create the non-standard hybrid by extracting the nonce from the NIST envelope while preserving its 0x29 nonce-less header. A correct conversion to detached form would replace 0x29 with 0x39.

Impact An attacker can change the first byte of a valid 0x39 signature to 0x29 without invalidating it. This produces multiple accepted byte encodings for the same signature and message.

This does not appear to enable a signature over a new message or public key. However, it has the following consequences:

- ▶ The verifier accepts signatures that do not conform to either defined Falcon framing.
- ▶ Signature bytes are malleable, which can affect systems that hash, identify, deduplicate, cache, or route signatures using their serialized representation.

- ▶ The permissive behavior can cause interoperability differences with strict Falcon implementations, whose detached verifiers reject 0x29.
- ▶ It creates ambiguity for future format dispatch, particularly if support for the 0x59 fixed-width encoding is added.
- ▶ Documentation incorrectly treats the behavior as necessary for signer interoperability.

Recommendation Require 0x39 for every compressed detached Falcon-512 signature:

```
1 if signature[0] != 0x39 {  
2     return false;  
3 }
```

Continue determining natural versus padded compressed form from decoder consumption and length:

- ▶ Accept exact consumption for a natural-length compressed signature.
- ▶ Accept trailing zeros only when the total signature length is exactly 666 bytes.
- ▶ Do not use 0x29 to identify padded signatures; padded signatures also use 0x39.

Developers Response The developers implemented the suggested recommendation. 0x39 is now required as the header for a detached signature layout, while header 0x29 is rejected.

5.1.2 V-FSA-VUL-002: Unsafe key rotation may permanently lock smart accounts

Severity	Low	Commit	79d8c99
Type	Access Control	Status	Fixed
Location(s)	contracts/soroban-falcon-smart-account/[...]/lib.rs:125-143		
Confirmed Fix At	stellar-pq/pull/3		

Description The key-rotation process of the `FalconSmartAccount` contract is a one-step process that only validates whether the new public key is of the required size, but does not verify whether it is a valid Falcon public key or, more importantly, whether it is the intended public key. Therefore, this process is not robust against potential mistakes during its execution.

Impact Setting an incorrect public key during key rotation is an irrecoverable mistake that renders the smart wallet unusable. This includes both a key other than the intended one and a corrupted copy of the intended key.

Recommendation Validate that `new_pubkey` is a well-formed Falcon-512 public key before storing it. A well-formedness check will reject malformed encodings, including some typos in an otherwise intended key.

A well-formed but incorrect public key will still pass that check. To address that case, use a two-step key-rotation process in which the current key authorizes `propose_key()`, storing `new_pubkey` as pending while the existing key remains active. The pending key should then authorize `accept_key()`, proving possession of the corresponding private key before activation. If this verification is impractical, `accept_key()` may instead require authorization from the current key as an additional confirmation step. Until activation, the current key should remain able to cancel or replace the pending rotation.

At minimum, document that the current implementation cannot recover from an incorrect key rotation and emphasize that operators must carefully verify the new public key before proceeding.

Developers Response The developers implemented the suggested recommendation. The current admin now has the ability to propose a new admin (public key) and to cancel such proposal. The proposed admin is the only one allowed to accept the proposal by going through the signature verification flow.

5.1.3 V-FSA-VUL-003: Non-standard Falcon input limits may impair interoperability

Severity	Low	Commit	79d8c99
Type	Data Validation	Status	Fixed
Location(s)	contracts/falcon-512-core/src/ ▶ lib.rs:24, 27, 36 ▶ verify.rs:318		
Confirmed Fix At	stellar-pq/pull/4		

Description The verifier applies several limits on message and signature size and coefficients, that do not agree with standard Falcon-512 encodings:

- ▶ **Message Length:** `FALCON_MAX_MESSAGE_SIZE` is set to 16,384 bytes in `contracts/falcon-512-core/src/lib.rs` line 36. The limit is enforced in `FalconVerifier::verify_512` and by the Soroban wrapper at `contracts/soroban-falcon-verifier/src/lib.rs`. Falcon itself does not impose this, hence this is an implementation-specific limit.
- ▶ **Signature Coefficients:** `decode_sig_compressed` in `contracts/falcon-512-core/src/verify.rs` line 318 rejects an `s_2` coefficient whose absolute value exceeds 2,047. Although coefficients fall within `[-2047, 2047]` with overwhelming probability, and the reference compressed encoder applies the same restriction, the configured squared-norm bound of 34,034,726 does not by itself exclude coefficients up to 5,833. The verifier consequently accepts only a subset of mathematically valid signature vectors.
- ▶ **Maximum Signature Length:** In `contracts/falcon-512-core/src/lib.rs` line 24 `FALCON_SIG_MAX_SIZE` is set to 666 bytes. While 666 bytes is the exact size of a padded Falcon-512 signature, the maximum length of a variable-length compressed signature is 752 bytes. Valid compressed signatures between 667 and 752 bytes are rejected before decoding.
- ▶ **Minimum signature length:** `FALCON_SIG_MIN_SIZE` is set to 42 bytes, based on a one-byte header, a 40-byte nonce, and one polynomial byte. A compressed Falcon-512 signature encodes 512 coefficients, each requiring at least nine bits. The actual minimum is therefore $1 + 40 + (512 * 9 / 8) = 617$ bytes. Inputs between 42 and 616 bytes pass the initial size check but are subsequently rejected during decoding.

Impact Messages longer than 16,384 bytes and otherwise valid compressed signatures exceeding the coefficient or size limits cannot be verified. This may cause interoperability or availability issues when signatures are produced by implementations operating over the complete supported range.

The permissive minimum-length check will not result in invalid signatures being accepted, but allows malformed inputs to proceed through additional decoding work before rejection.

Recommendation Align the message, coefficient, and signature-length limits with the Falcon specification. In particular, support messages of arbitrary length, compressed signatures up to 752 bytes, and all coefficients permitted by the verification rules. The minimum signature length should be increased to 617 bytes to reject impossible encodings before decoding. Where an implementation-specific restriction is intentionally retained, explicitly document the

deviation, the accepted range, the reason for introducing the restriction, and its interoperability implications.

Developers Response Implementation specific limitations on the message length, signature coefficients, minimum and maximum signature length have been aligned with the Falcon specification.

5.1.4 V-FSA-VUL-004: Undocumented NTT primitives representation invariants may cause incorrect computations

Severity	Warning	Commit	79d8c99
Type	Maintainability	Status	Fixed
Location(s)	contracts/falcon-512-core/src/ntt.rs:91		
Confirmed Fix At	stellar-pq/pull/5		

Description The NTT implementation represents field elements as u32 values in scalar helpers and as u16 values in polynomial arrays. A field element has multiple integer representatives congruent modulo Q , while its canonical representative lies in $[0, Q)$. The implementation also uses two polynomial domains: coefficient and evaluation and two field encodings: natural and Montgomery.

Most primitives in `ntt.rs` rely on specific range, domain, and encoding invariants. When these preconditions hold, the functions produce correct canonical outputs in the documented domain and encoding. These preconditions are not documented consistently.

Impact Non-canonical inputs may violate arithmetic bounds, resulting in overflow, incorrect values, or non-canonical outputs. Likewise, passing a polynomial in the wrong domain or Montgomery representation may produce values that are valid field elements but do not represent the intended computation. Future refactors or code reuse could therefore pass values with an unsupported range, domain, or encoding. Such misuse can potentially lead to erroneous verification decisions.

Recommendation For each of the functions document the conditions on the inputs and the resulting guarantees on the result in precisely. The following functions should clearly document their input and output invariants:

1. `field_add()` and `field_sub()` require canonical x and y using the same encoding, either natural or Montgomery. Their output is canonical and preserves that encoding.
2. `field_halve()` requires canonical x in either natural or Montgomery encoding. Its output is canonical and preserves the encoding.
3. `montgomery_mul` requires x and y so that $x*y$ is bounded by $2^{16}*Q$. The result will be canonical.
4. `ntt_forward` requires a to be a coefficient-domain polynomial with canonical entries using a consistent encoding, either natural or Montgomery. Its output is canonical, remains in the same encoding, and is in the evaluation domain.
5. `ntt_inverse` requires a to be an evaluation-domain polynomial with canonical entries using a consistent encoding, either natural or Montgomery. Its output is canonical, remains in the same encoding, and is in the coefficient domain.
6. `poly_to_montgomery` requires f to contain canonical entries in natural encoding. Its output is canonical and Montgomery-encoded. Because the operation is pointwise, it preserves the polynomial domain.
7. `poly_sub` requires f and g to contain canonical entries and to use the same polynomial domain and encoding. Its output is canonical and preserves the other two properties.
8. `poly_prepare_for_mul` requires h to be a coefficient-domain polynomial with canonical entries in natural encoding. Its output is canonical, Montgomery-encoded, and in the evaluation domain.

9. `poly_pointwise_mul()` requires `f` and `g` to be evaluation-domain polynomials with canonical entries. Its output is canonical and remains in the evaluation domain. The encoding will depend on the encoding of the inputs.

Developers Response Assumed conditions on input and resulting guarantees on output have been documented.

5.1.5 V-FSA-VUL-005: Maintainability and Documentation Findings

Severity	Warning	Commit	79d8c99
Type	Maintainability	Status	Fixed
Location(s)	contracts/falcon-512-core/src/verify.rs:370		
Confirmed Fix At	stellar-pq/pull/6		

Description Veridise analysts identified the following improvements to make the codebase more maintainable and clearly documented:

1. Within `hash_to_point()` in `falcon-512-core/src/verify.rs`, the code guarantees that `v` lies in $[0, Q)$, with `debug_assert!(v < Q)` serving as defense in depth against future regressions. Because this check is omitted from release builds, replacing it with a runtime assertion would preserve the safeguard in the on-chain verifier.
2. In `falcon-512-core/src/verify.rs`, `hash_to_point()` absorbs nonce and message into SHAKE256 without encoding their boundary, so distinct input pairs with the same concatenation are processed identically, for example, `[1, 1] || [2, 2]` and `[1] || [1, 2, 2]`. This ambiguity does not affect the current implementation because the only caller, `verify_512()`, always supplies a 40-byte nonce. However, `hash_to_point()` should document this fixed-length requirement to prevent future callers from using ambiguous inputs.
3. The `FalconSmartAccount` stores its 897-byte Falcon public key in contract instance storage, consuming approximately 1.4% of the 64 KiB entry limit before serialization and ledger-entry overhead. While this is acceptable for the current design, the storage footprint and shared limit should be documented so future extensions account for the remaining capacity before adding instance state.
4. Within `ntt_inverse()` in `falcon-512-core/src/ntt.rs`, the local declaration `let logn = 9` duplicates the existing `FALCON_512_LOGN` constant. Using the shared constant would make the parameter dependency explicit and prevent the values from diverging during future changes.
5. Within `__check_auth()` in `soroban-falcon-smart-account/src/lib.rs`, the length check on the stored public key duplicates validation already performed by `__constructor()` and `rotate_key()`, the only functions that write to `FALCON_PUBKEY_KEY`. The stored value is therefore guaranteed to have the expected size under the current design, making this check redundant unless it is intentionally retained as defense in depth against future storage changes.
6. In `falcon-512-core/src/verify.rs`, `decode_pubkey()` writes decoded coefficients to the mutable output buffer `h` as processing progresses. If decoding later fails, the function returns false after `h` may have been partially modified, leaving its contents unspecified. The current caller, `verify_512()`, immediately returns on failure and therefore does not use this state; however, `decode_pubkey()` should document that callers must discard `h` whenever decoding fails.
7. In `falcon-512-core/src/verify.rs`, `verify_raw_512()` is publicly accessible even though it relies on invariants established by `verify_512()`: `c0` must contain canonical coefficient-domain entries, `s2` must use the expected signed range, and `h` must be canonical, Montgomery-encoded, and in the evaluation domain. Calling it directly with malformed inputs may cause a panic or return true for values that do not represent a valid Falcon signature. Since this function is an internal verification primitive, its visibility should be restricted. The functions in `ntt.rs` should likewise use the narrowest visibility

required by their callers, making their internal status explicit and reducing the risk of misuse.

8. In `falcon-512-core/src/ntt.rs`, line 172 the value `ni` is recomputed on every invocation by halving `R` nine times, although the fixed Falcon-512 parameters make this value invariant: $ni = R / 512 \bmod Q = 128$. While this does not currently affect correctness, it introduces unnecessary arithmetic and control flow, obscures a protocol constant, and increases maintenance risk. Define `ni` as a constant within `falcon-512-core/src/lib.rs` and use it directly.
9. In `falcon-512-core/src/verify.rs`, `decode_pubkey()` checks public-key length to be exactly 896 bytes on line 250, matching 512 14-bit coefficients with no padding. Therefore, `acc_len` is always zero at the final check on line 279, making it ineffective regardless of `acc`. Remove it or replace it with `debug_assert_eq!(acc_len, 0)` to document the invariant.
10. `FalconSmartAccount` depends on `soroban-sdk 23.4.0`, where `Events::publish` is deprecated in favor of `#[contractevent]`. Both `__constructor` (line 103) and `rotate_key` (line 148) still call `env.events().publish((symbol_short!("falcon"), symbol_short!("init"|"rotate")), pubkey_hash)`. Replace those calls with `#[contractevent]` types so the events stay in the contract spec and the deprecation does not become a break on a later SDK bump.

Developers Response All suggestions for improvement have been addressed.

6.1 Methodology

One of the goals of the security assessment was to fuzz test Falcon Smart Account for Soroban to evaluate its behavior against a reference implementation. The Veridise security analysts selected the `falcon-rs` Rust crate@[1] as a reference implementation. The crate's API is similar to the original C reference implementation, and it has been audited.

The fuzzing campaign had two primary goals. First, the Veridise security analysts sought to identify inputs that could cause the implementation to panic or otherwise enter a crash state, as such behavior could potentially result in denial of service. Second, the analysts sought to evaluate functional correctness by comparing its signature-verification behavior with the selected reference implementation. In particular, the campaign tested whether both implementations agreed on the validity of a given signature and the associated verification inputs.

The Veridise security analysts conducted three independent fuzzing runs using AFL++. Each run used a single CPU and a custom fuzzing harness that wrapped the implementation and transformed mutated fuzzer input into appropriately sized inputs for its API.

The first run exercised the implementation directly with randomly generated, correctly sized inputs and tested whether any such input could cause a panic. The second and third runs performed differential fuzzing against the `falcon-rs` reference implementation. For these runs, the same verification input was provided to both implementations, and the harness required both signature verifiers to produce the same Boolean result. Any input for which one implementation accepted the signature while the other rejected it was treated as a potential bug and retained for subsequent triage.

The second run performed this differential check using randomly generated inputs of the sizes expected by the verification APIs. The third run instead exercised the differential property using valid inputs: the Veridise security analysts generated key pairs and signatures using the reference implementation's key-pair generation and signing APIs and then supplied the resulting verification inputs to both implementations. This provided coverage of valid signatures in addition to the arbitrary inputs explored during the second run.

6.2 Properties Fuzzed

The first property tested was panic resistance. For correctly sized arbitrary inputs accepted by the Falcon Smart Account for Soroban API, verification should not cause the implementation to panic.

The second property tested was consistency with the reference implementation. Given identical verification inputs, Falcon Smart Account for Soroban and the `falcon-rs` reference implementation should agree on whether a signature is valid. This property was tested both over arbitrary correctly sized inputs and over valid signatures generated using the reference implementation.

The Veridise security analysts devoted a total of 96 compute-hours to fuzzing Falcon Smart Account for Soroban. The differential fuzzing runs produced six unique crash inputs for

manual investigation: one during the arbitrary-input differential run and five during the valid-signature differential run. Manual triage determined that all six were false positives arising from implementation differences that do not affect the security of the system. Consequently, the fuzzing campaign identified no security issues.

Table 6.1: Summary of the Falcon Smart Account for Soroban fuzzing campaign.

Run	Duration	Property	Crashes	Confirmed Issues
1	24 h.	Panic resistance	0	0
2	36 h.	Differential fuzzing, arbitrary inputs	1	0
3	36 h.	Differential fuzzing, valid signatures	5	0
Total	96 h.		6	0



Bibliography

- [1] lattice-safe team. *falcon-rs*. <https://github.com/lattice-safe/falcon-rs>. 2026 (cited on page 18).